

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  int countZero(int a[], int l, int r) {
3      if (l > r) return 0;
4      int mid = (l + r) / 2;
5      if (a[mid] == 0)
6          return (r - mid + 1) + countZero(a, l, mid - 1);
7      return countZero(a, mid + 1, r);
8  }
9  int main() {
10     int m;
11     scanf("%d", &m);
12     int a[m];
13     for (int i = 0; i < m; i++)
14         scanf("%d", &a[i]);
15     printf("%d", countZero(a, 0, m - 1));
16     return 0;
17 }
```

	Input	Expected	Got	
✓	5	2	2	✓
	1			
	1			
	1			
	0			
	0			

	Input	Expected	Got	
✓	10 1 1 1 1 1 1 1 1 1 1 1 1	0	0	✓
✓	8 0 0 0 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main() {
3     int n, count = 0, major = 0;
4     scanf("%d", &n);
5     int a[n];
6     for(int i = 0; i < n; i++)
7         scanf("%d", &a[i]);
8     for(int i = 0; i < n; i++) {
9         if(count == 0)
10            major = a[i];
11         if(a[i] == major)
12             count++;
13         else
14             count--;
15     }
16     printf("%d", major);
17     return 0;
18 }
```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  int floorValue(int a[], int l, int r, int x) {
3      if (l > r)
4          return -1;
5      int mid = (l + r) / 2;
6      if (a[mid] == x)
7          return a[mid];
8      if (a[mid] < x)
9          return (floorValue(a, mid + 1, r, x) != -1) ?
10             floorValue(a, mid + 1, r, x) : a[mid];
11     return floorValue(a, l, mid - 1, x);
12 }
13 int main() {
14     int n, x;
15     scanf("%d", &n);
16     int a[n];
17     for(int i = 0; i < n; i++)
18         scanf("%d", &a[i]);
19     scanf("%d", &x);
20     printf("%d", floorValue(a, 0, n - 1, x));
21     return 0;
22 }
```

	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓

	Input	Expected	Got	
✓	5	85	85	✓
	10			
	22			
	85			
	108			
	129			
	100			
✓	7	9	9	✓
	3			
	5			
	7			
	9			
	11			
	13			
	15			
	10			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  int findPair(int a[], int l, int r, int x) {
3      if (l >= r) return 0;
4      if (a[l] + a[r] == x) {
5          printf("%d\n%d", a[l], a[r]);
6          return 1;
7      }
8      if (a[l] + a[r] < x)
9          return findPair(a, l + 1, r, x);
10     return findPair(a, l, r - 1, x);
11 }
12 int main() {
13     int n, x;
14     scanf("%d", &n);
15     int a[n];
16     for(int i = 0; i < n; i++)
17         scanf("%d", &a[i]);
18     scanf("%d", &x);
19     if (!findPair(a, 0, n - 1, x))
20         printf("No");
21     return 0;
22 }
```

	Input	Expected	Got	
✓	4 2 4 8 10 14	4 10	4 10	✓
✓	5 2 4 6 8 10 100	No	No	✓

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```

1  #include <stdio.h>
2
3  void quickSort(int a[], int low, int high) {
4      if (low >= high) return;
5
6      int i = low, j = high;
7      int pivot = a[(low + high) / 2];
8      int temp;
9
10     while (i <= j) {
11         while (a[i] < pivot) i++;
12         while (a[j] > pivot) j--;
13
14         if (i <= j) {
15             temp = a[i];
16             a[i] = a[j];
17             a[j] = temp;
18             i++;
19             j--;
20         }
21     }
22
23     if (low < j) quickSort(a, low, j);
24     if (i < high) quickSort(a, i, high);
25 }
26
27 int main() {
28     int n;
29     scanf("%d", &n);
30
31     int a[n];
32     for (int i = 0; i < n; i++)
33         scanf("%d", &a[i]);
34
35     quickSort(a, 0, n - 1);
36
37     for (int i = 0; i < n; i++)
38         printf("%d ", a[i]);
39
40     return 0;
41 }
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)